TASKFLOW

Görev ve Proje Yönetim Sistemi API Geliştirme Projesi

Proje Envanteri — Tasarım Çıktısı

Proje	TaskFlow — Görev ve Proje Yönetim Sistemi API'si
Doküman	Proje Envanteri (Proje Tanıtımı, Veri Modeli ve API Tasarımı)
Hazırlayan	Meryem Nisa Demir
Eğitim	Backend Geliştirme — Node.js ile Backend Programlama (Bitirme Projesi)
Program	Software Persona Yazılım Mesleki Gelişim Programı
GitHub	https://github.com/nisademir1/taskflow-api
Teknolojiler	Node.js, Express.js, REST API, JSON, Postman

1. Projenin Amacı

Bu projenin amacı, Node.js backend eğitiminde öğrenilen konuları (Node.js temelleri, Express.js ile sunucu geliştirme, routing, middleware, CRUD operasyonları ve JSON ile veri yönetimi) gerçek hayatta kullanılabilecek bir uygulama üzerinde göstermektir. Bunun için bir yazılım ekibinin görevlerini yönetebileceği, tamamen REST mantığıyla çalışan bir API geliştirilmiştir.

2. Senaryo

Bir yazılım şirketi, ekibindeki görevleri, projeleri ve çalışanların sorumluluklarını tek bir yerden takip etmek istemektedir. Görevler mesajlaşma uygulamaları ve tablolar arasında dağınık durumdadır; kimin hangi işle uğraştığı, hangi işin acil olduğu ve neyin bittiği net değildir. TaskFlow API bu sorunu şu şekilde çözer:

- Proje yöneticisi yeni bir görev oluşturur ve bir çalışana atar.
- Görevin önceliği (low / medium / high) ve son teslim tarihi belirlenir.
- Çalışan işe başladığında durumu **in-progress**, bitirdiğinde **completed** olarak günceller.
- Gereksiz veya iptal edilen görevler sistemden silinir.
- Sisteme gelen tüm istekler Logger middleware ile kayıt altına alınır.

API, ileride bir web veya mobil arayüzün bağlanabileceği bir altyapı olarak tasarlanmıştır.

3. Sistem Mimarisi

Proje katmanlı bir yapıda düzenlenmiştir; her katmanın tek bir sorumluluğu vardır. Bir isteğin sistem içindeki yolu:

```
İstek → express.json() → Logger → Router → (Validation) → Controller → fileDb → tasks.json
                                                                ↓
                                                          JSON yanıt
```

Katman	Dosya	Görevi
Giriş	server.js, src/app.js	Sunucuyu başlatır; middleware ve route'ları bağlar
Routing	src/routes/	Hangi adresin hangi fonksiyona gideceğini belirler
Controller	src/controllers/	İş mantığı: ekleme, listeleme, detay, güncelleme, silme
Middleware	src/middleware/	Logger (zorunlu), validation ve hata yönetimi

Katman	Dosya	Görevi
Veri	src/utils/fileDb.js, src/data/tasks.json	JSON dosyasına okuma ve yazma

4. Veri Modeli Tasarımı

Sistemin temel varlığı **Görev (Task)** nesnesidir. Veriler `src/data/tasks.json` dosyasında JSON dizisi olarak saklanır.

Alan	Tip	Zorunlu	Açıklama
id	number	Otomatik	Benzersiz, otomatik artan numara
title	string	Evet	Görev başlığı (en az 3 karakter)
description	string	Evet	Görev açıklaması
status	string	Hayır	pending, in-progress, completed (varsayılan: pending)
priority	string	Evet	low, medium, high
assignee	string	Evet	Görevin atandığı çalışan
project	string	Hayır	Görevin ait olduğu proje (varsayılan: Genel)
dueDate	string / null	Hayır	Son teslim tarihi (örn. 2026-11-15)
createdAt	string	Otomatik	Oluşturulma zamanı (ISO formatında)
updatedAt	string	Otomatik	Son güncellenme zamanı

Örnek görev nesnesi:

```
{
  "id": 3,
  "title": "Postman testlerini yap",
  "description": "Tüm CRUD uçları Postman ile test edilecek",
  "status": "pending",
  "priority": "high",
  "assignee": "Nisa",
  "project": "TaskFlow",
  "dueDate": "2026-11-10",
  "createdAt": "2026-11-01T19:22:14.766Z",
  "updatedAt": "2026-11-01T19:22:14.766Z"
}
```

5. API Tasarımı — Endpoint Listesi

Temel adres: `http://localhost:3000`. Tüm yanıtlar JSON formatındadır ve `success` alanı içerir.

Zorunlu — CRUD operasyonları

İşlem	Method	Endpoint	Başarılı yanıt	Hata durumları
Görev ekleme	POST	/api/tasks	201 Created	400 (geçersiz veri)
Görev listeleme	GET	/api/tasks	200 OK	—
Görev detayı	GET	/api/tasks/:id	200 OK	404 (bulunamadı)
Görev güncelleme	PUT	/api/tasks/:id	200 OK	400, 404
Görev silme	DELETE	/api/tasks/:id	200 OK	404 (bulunamadı)

İsteğe bağlı — Gelişmiş özellikler

Özellik	Örnek istek
Duruma / önceliğe / kişiye göre filtreleme	GET /api/tasks?status=completed · ?priority=high
Görev arama	GET /api/tasks/search?keyword=backend
Kişiye göre listeleme	GET /api/tasks?assignee=musa
Sayfalama	GET /api/tasks?page=1&limit=10
Raporlama	GET /api/reports/completed · /pending · /summary

6. Middleware Yapısı

- **Logger (zorunlu):** Her isteğin method, endpoint, zaman damgası, durum kodu ve süresini konsola ve `logs/requests.log` dosyasına yazar. Örnek: `[2026-11-01T18:51:24.798Z] POST /api/tasks → 201 (3 ms)`
- **Validation (isteğe bağlı):** title, description, priority ve assignee alanlarını doğrular; geçersiz veride açıklayıcı mesajla 400 Bad Request döndürür.
- **Hata yönetimi:** Olmayan adreslere 404, bozuk JSON'a 400, beklenmeyen hatalara 500 döndürür.

7. Test Süreci

Tüm CRUD uçları Postman ile sırasıyla test edilmiş ve beklenen sonuçlar alınmıştır. Test ekran görüntüleri ve Postman koleksiyonu GitHub reposundaki `docs/` klasöründedir.

#	Test	İstek	Beklenen	Sonuç
1	Görev ekleme	POST /api/tasks	201 Created	Başarılı
2	Görev listeleme	GET /api/tasks	200 OK, count: 3	Başarılı
3	Görev detayı	GET /api/tasks/3	200 OK	Başarılı
4	Görev güncelleme	PUT /api/tasks/3	200 OK, status: completed	Başarılı
5	Görev silme	DELETE /api/tasks/3	200 OK	Başarılı